DBMS PROJECT ON

MEDICAL *STORE MANAGEMENT SYSTEM*

Shaik Thamanna-25WH1A05A2 https://github.com/25wh1a05a2
V. Namratha-25WH1A05A9
Umme Habeeba-25WH1A05A5
V.Bhavani-25WH1A05B7
Sheetal Chavan-26WH5A0512

1.ER-DIAGRAM FOR MEDICAL STORE MANAGEMENT SYSTEM

The Medical Store Management System is designed to manage the daily activities of a medical shop. It maintains information about medicines, suppliers, purchases, customers, employees and sales.

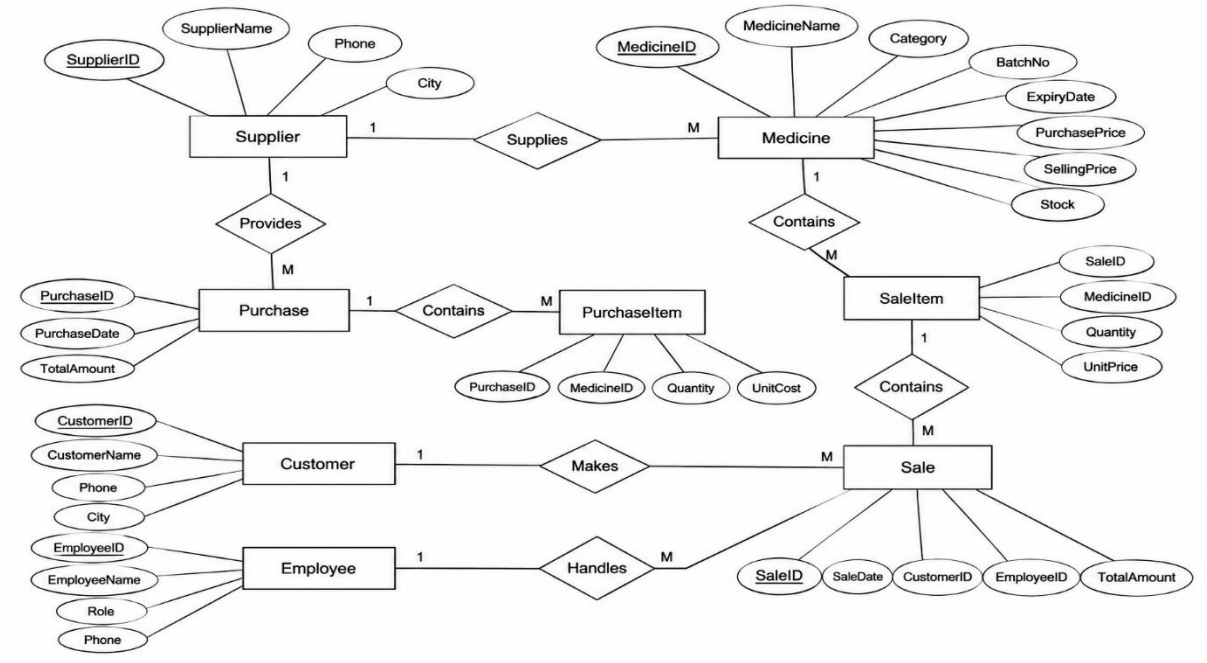
The system helps to manage medicine stock, purchase details, sales transactions and customer information efficiently. It also maintains medicine expiry dates, batch numbers, purchase prices and selling prices.

The Medical Store Management System consists of eight entities connected through different relationships.

- A Supplier supplies multiple medicines to the medical store.
- A Supplier can provide multiple purchases.
- Each Purchase contains multiple PurchaseItems.
- Each PurchaseItem refers to a particular medicine.
- A Customer can make multiple sales.
- An Employee handles multiple sales.
- Each Sale contains multiple SaleItems.
- Each SaleItem refers to a particular medicine.

The PurchaseItem and SaleItem entities are used to maintain the quantity and price of each medicine in a purchase or sales transaction.

The Medicine entity maintains stock details, batch numbers and expiry dates to help manage medicine availability.



2.RELATIONAL MODEL

A relational model represents data in the form of tables called relations. Each table contains rows and columns.

1. Supplier Table

Attribute	Key	Description
SupplierID	Primary Key	Unique supplier ID
SupplierName	-	Name of supplier
Phone	-	Supplier phone number
City	-	Supplier city

2. Medicine Table

Attribute	Key	Description
MedicineID	Primary Key	Unique medicine ID
MedicineName	-	Name of medicine
Category	-	Medicine category
BatchNo	-	Medicine batch number
ExpiryDate	-	Expiry date
PurchasePrice	-	Purchase price
SellingPrice	-	Selling price
Stock	-	Available stock
SupplierID	Foreign Key	Supplier ID

3. Purchase Table

Attribute	Key	Description
PurchaseID	Primary Key	Unique purchase ID
PurchaseDate	-	Date of purchase
TotalAmount	-	Total purchase amount
SupplierID	Foreign Key	Supplier ID

4. PurchaseItem Table

Attribute	Key	Description
PurchaseID	Composite PK, FK	Purchase ID
MedicineID	Composite PK, FK	Medicine ID
Quantity	-	Purchased quantity
UnitCost	-	Cost per unit

5. Customer Table

Attribute	Key	Description
CustomerID	Primary Key	Unique customer ID
CustomerName	-	Customer name
Phone	-	Customer phone
City	-	Customer city

6. Employee Table

Attribute	Key	Description
EmployeeID	Primary Key	Unique employee ID
EmployeeName	-	Employee name
Role	-	Employee designation
Phone	-	Employee phone

7. Sale Table

Attribute	Key	Description
SaleID	Primary Key	Unique sale ID
SaleDate	-	Date of sale
CustomerID	Foreign Key	Customer ID
EmployeeID	Foreign Key	Employee ID
TotalAmount	-	Total sale amount

8. SaleItem Table

Attribute	Key	Description
SaleID	Composite PK, FK	Sale ID
MedicineID	Composite PK, FK	Medicine ID
Quantity	-	Sold quantity
UnitPrice	-	Selling price per unit

3: NORMALIZATION

Normalization is the process of organizing data in a database to reduce data redundancy and avoid insertion, deletion and update anomalies.

There are different normal forms:

1. First Normal Form (1NF)
2. Second Normal Form (2NF)
3. Third Normal Form (3NF)
4. Boyce-Codd Normal Form (BCNF)
5. Fifth Normal Form (5NF)
6. Sixth Normal Form (6NF)

First Normal Form (1NF)

A relation is in 1NF if every column contains atomic (single) values and there are no repeating groups.

Example – Before 1NF

SaleID	CustomerName	Medicines
501	Anu	Paracetamol, Cetirizine
502	Ravi	Vitamin C, ORS

After 1NF

SaleID	CustomerName	Medicine
501	Anu	Paracetamol
501	Anu	Cetirizine
502	Ravi	Vitamin C
502	Ravi	ORS

Second Normal Form (2NF)

A relation is in 2NF if it is already in 1NF and has no partial dependency.

A partial dependency occurs when a non-key attribute depends on only part of a composite primary key.

Example – Before 2NF

SaleItem(SaleID, MedicineID, SaleDate, MedicineName, Quantity)

Composite Primary Key: (SaleID, MedicineID)

Dependencies:

- SaleID → SaleDate
- MedicineID → MedicineName
- (SaleID, MedicineID) → Quantity

After 2NF

Sale Table

SaleID (PK)	SaleDate
501	2026-11-01
502	2026-11-02

Medicine Table

MedicineID (PK)	MedicineName
101	Paracetamol
102	Cetirizine

SaleItem Table

SaleID (PK, FK)	MedicineID (PK, FK)	Quantity
501	101	2
501	102	1

Third Normal Form (3NF)

A relation is in 3NF if it is in 2NF and contains no transitive dependency.

A transitive dependency occurs when a non-key attribute depends on another non-key attribute.

Example – Before 3NF

EmployeeID	EmployeeName	DepartmentID	DepartmentName
301	Asha	D1	Pharmacy
302	Balu	D2	Billing

Dependencies:

- EmployeeID → DepartmentID
- DepartmentID → DepartmentName

After 3NF

Employee Table

EmployeeID	EmployeeName	DepartmentID
301	Asha	D1
302	Balu	D2

Department Table

DepartmentID	DepartmentName
D1	Pharmacy
D2	Billing

Boyce-Codd Normal Form (BCNF)

A relation is in BCNF if for every non-trivial functional dependency X → Y, X must be a superkey.

BCNF is a stronger version of 3NF.

Example :Consider the relation:

ClassAllocation(Student, Course, Instructor)

Functional dependencies:

- (Student, Course) → Instructor
- Instructor → Course

Therefore, the relation violates BCNF.

After BCNF decomposition

InstructorCourse

Instructor (PK)	Course
Ravi	DBMS
Kumar	Java

StudentInstructor

Student	Instructor
Anu	Ravi
Balu	Kumar

Fifth Normal Form (5NF)

A relation is in 5NF if it is in 4NF and has no non-trivial join dependencies other than those implied by candidate keys.

It removes redundancy caused by complex join dependencies.

Example:Consider:

SupplierPartProject(Supplier, Part, Project)It can be decomposed into:

SupplierPart

Supplier	Part
S1	P1
S2	P2

SupplierProject

Supplier	Project
S1	J1
S2	J2

PartProject

Part	Project
P1	J1
P2	J2

This decomposition is valid only when the business rules guarantee that the original relation can be reconstructed from these three relations without introducing incorrect combinations.

Sixth Normal Form (6NF)

A relation is in 6NF if it has no non-trivial join dependencies. It is mainly used in temporal databases.

Example:Consider employee information:

Employee

EmployeeID	EmployeeName
301	Asha

EmployeeRole

EmployeeID	ValidFrom	Role
301	2025-01-01	Pharmacist

EmployeePhone

EmployeeID	ValidFrom	Phone
301	2025-01-01	9100000001

4.PRACTICING DDL COMMANDS

mysql> CREATE DATABASE MedicalStoreDB;

OUTPUT:Query OK, 1 row affected

mysql> USE MedicalStoreDB;

OUTPUT:Database changed

mysql> SHOW TABLES;

```
+--------------------------+
| Tables_in_medicalstoredb|
+--------------------------+
| customer                 |
| employee                 |
| medicine                 |
| purchase                 |
| purchaseitem             |
| sale                     |
| saleitem                 |
| supplier                 |
+--------------------------+
8 rows in set
```

```
CREATE TABLE Supplier (
   SupplierID INT PRIMARY KEY,
   SupplierName VARCHAR(80),
   Phone VARCHAR(15),
   City VARCHAR(50)
);

CREATE TABLE Medicine (
   MedicineID INT PRIMARY KEY,
```

```
    MedicineName VARCHAR(100),
    Category VARCHAR(50),
    BatchNo VARCHAR(30),
    ExpiryDate DATE,
    PurchasePrice DECIMAL(10,2),
    SellingPrice DECIMAL(10,2),
    Stock INT,
    SupplierID INT,
    FOREIGN KEY (SupplierID)
    REFERENCES Supplier(SupplierID)
);

CREATE TABLE Purchase (
    PurchaseID INT PRIMARY KEY,
    PurchaseDate DATE,
    TotalAmount DECIMAL(10,2),
    SupplierID INT,
    FOREIGN KEY (SupplierID)
    REFERENCES Supplier(SupplierID)
);

CREATE TABLE PurchaseItem (
    PurchaseID INT,
    MedicineID INT,
    Quantity INT,
    UnitCost DECIMAL(10,2),
    PRIMARY KEY(PurchaseID, MedicineID),
    FOREIGN KEY(PurchaseID)
    REFERENCES Purchase(PurchaseID),
    FOREIGN KEY(MedicineID)
    REFERENCES Medicine(MedicineID)
);

CREATE TABLE Customer (
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(80),
    Phone VARCHAR(15),
    City VARCHAR(50)
);

CREATE TABLE Employee (
    EmployeeID INT PRIMARY KEY,
    EmployeeName VARCHAR(80),
    Role VARCHAR(40),
    Phone VARCHAR(15)
);

CREATE TABLE Sale (
    SaleID INT PRIMARY KEY,
    SaleDate DATE,
    CustomerID INT,
    EmployeeID INT,
    TotalAmount DECIMAL(10,2),
    FOREIGN KEY(CustomerID)
    REFERENCES Customer(CustomerID),
```

```
    FOREIGN KEY(EmployeeID)
    REFERENCES Employee(EmployeeID)
);

CREATE TABLE SaleItem (
    SaleID INT,
    MedicineID INT,
    Quantity INT,
    UnitPrice DECIMAL(10,2),
    PRIMARY KEY(SaleID, MedicineID),
    FOREIGN KEY(SaleID)
    REFERENCES Sale(SaleID),
    FOREIGN KEY(MedicineID)
    REFERENCES Medicine(MedicineID)
);
```

OUTPUT:

```
Query OK, 0 rows affected
```

ALTER:

```
ALTER TABLE Customer
ADD Email VARCHAR(100);
```

OUTPUT:

```
Query OK, 0 rows affected
```

DROP:

```
ALTER TABLE Customer
DROP COLUMN Email;
```

OUTPUT:

```
Query OK, 0 rows affected
```

5.PRACTICING DDL COMMANDS

INSERT:

SUUPLIER:

```
INSERT INTO Supplier VALUES
(1,'MediSource','9876500001','Hyderabad'),
(2,'HealthCare','9876500002','Vijayawada'),
(3,'LifeLine','9876500003','Warangal'),
(4,'PrimeMed','9876500004','Guntur'),
(5,'Aarogya','9876500005','Hyderabad'),
(6,'CityPharma','9876500006','Nellore'),
(7,'Wellness','9876500007','Karimnagar'),
(8,'CarePlus','9876500008','Tirupati'),
(9,'MedWorld','9876500009','Khammam'),
(10,'TrustPharma','9876500010','Hyderabad');
```

MEDICINE

```
 INSERT INTO Medicine VALUES
(101,'Paracetamol','Tablet','B101','2027-12-31',1.50,2.00,200,1),
(102,'Cetirizine','Tablet','B102','2027-10-31',2.00,3.00,150,2),
(103,'Amoxicillin','Capsule','B103','2027-08-31',4.00,5.50,100,3),
(104,'Cough Syrup','Syrup','B104','2027-06-30',35.00,45.00,60,4),
(105,'Vitamin C','Tablet','B105','2028-01-31',3.00,4.00,120,5),
(106,'Antacid Gel','Gel','B106','2027-09-30',28.00,36.00,45,6),
(107,'ORS','Powder','B107','2028-03-31',8.00,10.00,90,7),
(108,'Ibuprofen','Tablet','B108','2027-11-30',2.50,3.50,110,8),
(109,'Metformin','Tablet','B109','2028-02-29',3.50,4.50,80,9),
(110,'Antiseptic Cream','Cream','B110','2027-07-31',22.00,29.00,50,10)
```

CUSTOMER

```
INSERT INTO Customer VALUES
(201,'Anu','9000000001','Hyderabad'),
(202,'Ravi','9000000002','Warangal'),
(203,'Meena','9000000003','Vijayawada'),
(204,'Kiran','9000000004','Guntur'),
(205,'Sita','9000000005','Hyderabad'),
(206,'Arjun','9000000006','Nellore'),
(207,'Divya','9000000007','Tirupati'),
(208,'Rahul','9000000008','Khammam'),
(209,'Pooja','9000000009','Karimnagar'),
(210,'Naveen','9000000010','Hyderabad');
```

EMPLOYEE

```
INSERT INTO Employee VALUES
(301,'Asha','Pharmacist','9100000001'),
(302,'Balu','Cashier','9100000002'),
(303,'Charan','Pharmacist','9100000003'),
(304,'Deepa','Manager','9100000004'),
(305,'Esha','Cashier','9100000005'),
(306,'Farhan','Pharmacist','9100000006'),
(307,'Geetha','Cashier','9100000007'),
(308,'Hari','Pharmacist','9100000008'),
(309,'Isha','Cashier','9100000009'),
(310,'Jai','Manager','9100000010');
```

PURCHASE

```
INSERT INTO Purchase VALUES
(401,'2026-10-01',300,1),
(402,'2026-10-02',250,2),
(403,'2026-10-03',400,3),
(404,'2026-10-04',350,4),
(405,'2026-10-05',360,5),
(406,'2026-10-06',280,6),
(407,'2026-10-07',320,7),
(408,'2026-10-08',275,8),
(409,'2026-10-09',310,9),
(410,'2026-10-10',290,10);
```

PURCHASEITEM

```
INSERT INTO PurchaseItem VALUES
(401,101,100,1.50),
(402,102,80,2.00),
(403,103,60,4.00),
(404,104,10,35.00),
(405,105,70,3.00),
(406,106,10,28.00),
(407,107,30,8.00),
(408,108,50,2.50),
(409,109,40,3.50),
(410,110,10,22.00);
```

SALE

```
INSERT INTO Sale VALUES
(501,'2026-11-01',201,301,10),
(502,'2026-11-01',202,302,15),
(503,'2026-11-02',203,303,11),
(504,'2026-11-02',204,304,45),
```

(505,'2026-11-03',205,305,20),
(506,'2026-11-03',206,306,36),
(507,'2026-11-04',207,307,20),
(508,'2026-11-04',208,308,17.50),
(509,'2026-11-05',209,309,22.50),
(510,'2026-11-05',210,310,29);

SALEITEM

INSERT INTO SaleItem VALUES
(501,101,5,2.00),
(502,102,5,3.00),
(503,103,2,5.50),
(504,104,1,45.00),
(505,105,5,4.00),
(506,106,1,36.00),
(507,107,2,10.00),
(508,108,5,3.50),
(509,109,5,4.50),
(510,110,1,29.00);

OUTPUT:

```
Query OK, 0 rows affected
```

SELECT:

SELECT * FROM Medicine;

OUTPUT:

```
+------------+------------------+----------+--------+
| MedicineID | MedicineName     | Category | Stock  |
+------------+------------------+----------+--------+
| 101        | Paracetamol      | Tablet   | 200    |
| 102        | Cetirizine       | Tablet   | 150    |
| 103        | Amoxicillin      | Capsule  | 100    |
| 104        | Cough Syrup      | Syrup    | 60     |
| 105        | Vitamin C        | Tablet   | 120    |
| 106        | Antacid Gel      | Gel      | 45     |
| 107        | ORS              | Powder   | 90     |
| 108        | Ibuprofen        | Tablet   | 110    |
| 109        | Metformin        | Tablet   | 80     |
| 110        | Antiseptic Cream | Cream    | 50     |
+------------+------------------+----------+--------+
10 rows in set
```

UPDATE:

UPDATE Medicine
SET SellingPrice = 2.50
WHERE MedicineID = 101;

OUTPUT:

```
Query OK, 0 rows affected
```

DELETE:

DELETE FROM SaleItem
WHERE SaleID = 510 AND MedicineID = 110;

OUTPUT:

```
Query OK, 0 rows affected
```

COUNT:

SELECT COUNT(*) AS TotalMedicines
FROM Medicine;

OUTPUT:

```
+----------------+
| TotalMedicines |
+----------------+
| 10             |
+----------------+
1 row in set
```

6.SQL queries using any,all,union,intersect,join and constraints of the medical store management system.

ANY:

```
SELECT MedicineName, SellingPrice
FROM Medicine
WHERE SellingPrice > ANY
(SELECT SellingPrice
 FROM Medicine
 WHERE Category='Tablet');
```

Output:

```
+------------------+-------------+
| MedicineName     | SellingPrice|
+------------------+-------------+
| Amoxicillin      | 5.50        |
| Cough Syrup      | 45.00       |
| Antacid Gel      | 36.00       |
| ORS              | 10.00       |
| Antiseptic Cream | 29.00       |
+------------------+-------------+
```

ALL:

```
SELECT MedicineName, SellingPrice
FROM Medicine
WHERE SellingPrice >= ALL
(SELECT SellingPrice FROM Medicine);
```

OUTPUT:

```
+--------------+-------------+
| MedicineName | SellingPrice|
+--------------+-------------+
| Cough Syrup  | 45.00       |
+--------------+-------------+
1 row in set
```

UNION:

```
SELECT City FROM Supplier
UNION
SELECT City FROM Customer;
```

OUTPUT:

```
+------------+
| City       |
+------------+
| Hyderabad  |
| Vijayawada |
| Warangal   |
| Guntur     |
| Nellore    |
| Karimnagar |
| Tirupati   |
| Khammam    |
+------------+
8 rows in set
```

INTERSECT:

```
SELECT City FROM Supplier
INTERSECT
SELECT City FROM Customer;
```

OUTPUT:

```
+-----------+
| City      |
+-----------+
| Hyderabad |
| Vijayawada|
| Warangal  |
| Guntur    |
| Nellore   |
| Karimnagar|
| Tirupati  |
| Khammam   |
+-----------+
8 rows in set
```

JOIN:

```
SELECT Sale.SaleID,
       Customer.CustomerName,
       Employee.EmployeeName,
       Sale.TotalAmount
FROM Sale
JOIN Customer
ON Sale.CustomerID = Customer.CustomerID
JOIN Employee
ON Sale.EmployeeID = Employee.EmployeeID;
```

OUTPUT:

```
+--------+--------------+--------------+-------------+
| SaleID | CustomerName | EmployeeName | TotalAmount |
+--------+--------------+--------------+-------------+
| 501    | Anu          | Asha         | 10.00       |
| 502    | Ravi         | Balu         | 15.00       |
| 503    | Meena        | Charan       | 11.00       |
| 504    | Kiran        | Deepa        | 45.00       |
| 505    | Sita         | Esha         | 20.00       |
| 506    | Arjun        | Farhan       | 36.00       |
| 507    | Divya        | Geetha       | 20.00       |
| 508    | Rahul        | Hari         | 17.50       |
| 509    | Pooja        | Isha         | 22.50       |
| 510    | Naveen       | Jai          | 29.00       |
+--------+--------------+--------------+-------------+
10 rows in set
```

CONSTRAINTS:

```
CREATE TABLE MedicineTest (
   MedicineID INT PRIMARY KEY,
   MedicineName VARCHAR(50) NOT NULL,
   Price DECIMAL(10,2) CHECK (Price > 0),
   Stock INT DEFAULT 0
);
```

OUTPUT:

```
Query OK, 0 rows affected
```

NESTED QUERIES:

```
SELECT MedicineName, SellingPrice
FROM Medicine
```

```
WHERE SellingPrice >
(SELECT AVG(SellingPrice)
 FROM Medicine);
```

OUTPUT:

```
+------------------+-------------+
| MedicineName     | SellingPrice|
+------------------+-------------+
| Cough Syrup      | 45.00       |
| Antacid Gel      | 36.00       |
| ORS              | 10.00       |
| Antiseptic Cream | 29.00       |
+------------------+-------------+
4 rows in set
```

CORRELEATED SUBQUERY:

```
SELECT M.MedicineName,
       M.Category,
       M.SellingPrice
FROM Medicine M
WHERE M.SellingPrice >
(SELECT AVG(M2.SellingPrice)
 FROM Medicine M2
 WHERE M2.Category = M.Category);
```

OUTPUT:

```
Empty set
```

7.OUERIES USING AGGREGATE FUNCTIONS,WINDOW FUNCTIONS,GROUP BY,HAVING AND CREATIONAND DROPPING OF VIEWS

AGGREGATE FUNCTIONS:

COUNT():

```
SELECT COUNT(*) AS TotalMedicines
FROM Medicine;
```

OUTPUT:

```
+----------------+
| TotalMedicines |
+----------------+
| 10             |
+----------------+
```

SUM():

```
SELECT SUM(Stock) AS TotalStock
FROM Medicine;
```

OUTPUT:

```
+------------+
| TotalStock |
+------------+
| 1005       |
+------------+
```

AVG()

```
SELECT AVG(SellingPrice) AS AveragePrice
FROM Medicine;
```

OUTPUT:

```
+--------------+
| AveragePrice |
+--------------+
| 13.400000    |
+--------------+
```

MAX()

```
SELECT MAX(SellingPrice) AS HighestPrice
FROM Medicine;
```

OUTPUT:

```
+--------------+
| HighestPrice |
+--------------+
| 45.00        |
+--------------+
```

MIN():

```
SELECT MIN(SellingPrice) AS LowestPrice
FROM Medicine;
```

OUTPUT:

```
+-------------+
| LowestPrice |
+-------------+
| 2.00        |
+-------------+
```

GROUP BY:

```
SELECT Category,
    COUNT(*) AS TotalMedicines
FROM Medicine
GROUP BY Category;
```

OUTPUT:

```
+----------+----------------+
| Category | TotalMedicines |
+----------+----------------+
| Tablet   | 5              |
| Capsule  | 1              |
| Syrup    | 1              |
| Gel      | 1              |
| Powder   | 1              |
| Cream    | 1              |
+----------+----------------+
6 rows in set
```

HAVING:

```
SELECT Category,
    COUNT(*) AS TotalMedicines
FROM Medicine
GROUP BY Category
HAVING COUNT(*) > 1;
```

OUTPUT:

```
+----------+----------------+
| Category | TotalMedicines |
+----------+----------------+
| Tablet   | 5              |
+----------+----------------+
1 row in set
```

WINDOW FUNCTIONS:

```
SELECT MedicineName,
    SellingPrice,
    RANK() OVER
    (ORDER BY SellingPrice DESC) AS PriceRank
FROM Medicine;
```

OUTPUT:

```
+------------------+-------------+-----------+
| MedicineName     | SellingPrice| PriceRank |
+------------------+-------------+-----------+
| Cough Syrup      | 45.00       | 1         |
| Antacid Gel      | 36.00       | 2         |
| Antiseptic Cream | 29.00       | 3         |
| ORS              | 10.00       | 4         |
| Amoxicillin      | 5.50        | 5         |
| Metformin        | 4.50        | 6         |
| Vitamin C        | 4.00        | 7         |
| Ibuprofen        | 3.50        | 8         |
| Cetirizine       | 3.00        | 9         |
| Paracetamol      | 2.50        | 10        |
+------------------+-------------+-----------+
10 rows in set
```

CREATION OF VIEWS:

```
CREATE VIEW MedicineView AS
SELECT MedicineID,
    MedicineName,
    Category,
    Stock,
    SellingPrice
FROM Medicine;
SELECT * FROM MedicineView;
```

OUTPUT:

```
+------------+------------------+----------+-------+-------------+
| MedicineID | MedicineName     | Category | Stock | SellingPrice|
+------------+------------------+----------+-------+-------------+
| 101        | Paracetamol      | Tablet   | 200   | 2.00        |
| 102        | Cetirizine       | Tablet   | 150   | 3.00        |
| 103        | Amoxicillin      | Capsule  | 100   | 5.50        |
| 104        | Cough Syrup      | Syrup    | 60    | 45.00       |
| 105        | Vitamin C        | Tablet   | 120   | 4.00        |
| 106        | Antacid Gel      | Gel      | 45    | 36.00       |
| 107        | ORS              | Powder   | 90    | 10.00       |
| 108        | Ibuprofen        | Tablet   | 110   | 3.50        |
| 109        | Metformin        | Tablet   | 80    | 4.50        |
| 110        | Antiseptic Cream | Cream    | 50    | 29.00       |
+------------+------------------+----------+-------+-------------+
10 rows in set
```

8.TRIGGERS

BEFORE INSERT TRIGGER:

```
DELIMITER //
CREATE TRIGGER CheckStock
BEFORE INSERT ON SaleItem
FOR EACH ROW
BEGIN
```

```
  DECLARE AvailableStock INT;
    SELECT Stock INTO AvailableStock
    FROM Medicine
    WHERE MedicineID = NEW.MedicineID;
    IF AvailableStock IS NULL
      OR AvailableStock < NEW.Quantity THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT =
      'Insufficient Medicine Stock';
    END IF;
END //
DELIMITER ;
AFTER INSERT TRIGGER:
DELIMITER //
CREATE TRIGGER UpdateStock
AFTER INSERT ON SaleItem
FOR EACH ROW
BEGIN
    UPDATE Medicine
    SET Stock = Stock - NEW.Quantity
    WHERE MedicineID = NEW.MedicineID;
END //
DELIMITER ;
AFTER INSERT TRIGGER FOR PURCHASE:
DELIMITER //
CREATE TRIGGER PurchaseStock
AFTER INSERT ON PurchaseItem
FOR EACH ROW
BEGIN
    UPDATE Medicine
    SET Stock = Stock + NEW.Quantity
    WHERE MedicineID = NEW.MedicineID;
END //
DELIMITER ;
OUTPUT:
mysql> SHOW TRIGGERS;


+--------------+------------+-------------+
| Trigger      | Event      | Table       |
+--------------+------------+-------------+
| CheckStock   | INSERT     | saleitem    |
| UpdateStock  | INSERT     | saleitem    |
| PurchaseStock| INSERT     | purchaseitem|
+--------------+------------+-------------+
3 rows in set
```

TESTING THE TRIGGER:

```
INSERT INTO Sale
VALUES (511,'2026-11-06',201,301,4.00);
INSERT INTO SaleItem
VALUES (511,101,2,2.00);
MedicineID, MedicineName, Stock
FROM Medicine
WHERE MedicineID=101;
```

OUTPUT:

```
+------------+----------------+-------+
| MedicineID | MedicineName   | Stock |
+------------+----------------+-------+
| 101        | Paracetamol    | 198   |
+------------+----------------+-------+
1 row in set
```

9.PROCEDURES

DISPLAY ALL MEDICINES:

```
DELIMITER //
CREATE PROCEDURE GetMedicines()
BEGIN
   SELECT * FROM Medicine;
END //
DELIMITER ;
CALL GetMedicines();
```

OUTPUT:

```
+------------+------------------+----------+
| MedicineID | MedicineName     | Category |
+------------+------------------+----------+
| 101        | Paracetamol      | Tablet   |
| 102        | Cetirizine       | Tablet   |
| 103        | Amoxicillin      | Capsule  |
| 104        | Cough Syrup      | Syrup    |
| 105        | Vitamin C        | Tablet   |
| 106        | Antacid Gel      | Gel      |
| 107        | ORS              | Powder   |
| 108        | Ibuprofen        | Tablet   |
| 109        | Metformin        | Tablet   |
| 110        | Antiseptic Cream | Cream    |
+------------+------------------+----------+
10 rows in set
```

SEARCH MEDICINE BY ID:

```
DELIMITER //
CREATE PROCEDURE SearchMedicine(
   IN M_ID INT
)
BEGIN
   SELECT MedicineID,
        MedicineName,
        Stock,
        SellingPrice
   FROM Medicine
   WHERE MedicineID = M_ID;
END //
DELIMITER ;
CALL SearchMedicine(101);
```

OUTPUT:

```
+------------+--------------+-------+-------------+
| MedicineID | MedicineName | Stock | SellingPrice|
+------------+--------------+-------+-------------+
| 101        | Paracetamol  | 200   | 2.00        |
+------------+--------------+-------+-------------+
1 row in set
```

DISPLAY SALE DETAILS:

```
DELIMITER //
CREATE PROCEDURE SaleDetails(
   IN S_ID INT
)
BEGIN
   SELECT S.SaleID,
         S.SaleDate,
         C.CustomerName,
         E.EmployeeName,
         S.TotalAmount
   FROM Sale S
   JOIN Customer C
   ON S.CustomerID = C.CustomerID
   JOIN Employee E
   ON S.EmployeeID = E.EmployeeID
   WHERE S.SaleID = S_ID;
END //
DELIMITER ;
CALL SaleDetails(501);
```

OUTPUT:

```
+--------+------------+--------------+--------------+-------------+
| SaleID | SaleDate   | CustomerName | EmployeeName | TotalAmount |
+--------+------------+--------------+--------------+-------------+
| 501    | 2026-11-01 | Anu          | Asha         | 10.00       |
+--------+------------+--------------+--------------+-------------+
1 row in set
```

10.USAGE OF CURSORS

```
DELIMITER //
CREATE PROCEDURE LowStockMedicines()
BEGIN
   DECLARE Finished INT DEFAULT 0;
   DECLARE M_ID INT;
   DECLARE M_Name VARCHAR(100);
   DECLARE M_Stock INT;
   DECLARE MedicineCursor CURSOR FOR
      SELECT MedicineID, MedicineName, Stock
      FROM Medicine
      WHERE Stock < 100;
   DECLARE CONTINUE HANDLER
   FOR NOT FOUND SET Finished = 1;
   CREATE TEMPORARY TABLE LowStockResult(
      MedicineID INT,
      MedicineName VARCHAR(100),
      Stock INT
   );
   OPEN MedicineCursor;
   ReadLoop: LOOP
      FETCH MedicineCursor
      INTO M_ID, M_Name, M_Stock;
      IF Finished = 1 THEN
         LEAVE ReadLoop;
      END IF;
       INSERT INTO LowStockResult
```

```
        VALUES (M_ID, M_Name, M_Stock);
    END LOOP;
    CLOSE MedicineCursor;
    SELECT * FROM LowStockResult;
    DROP TEMPORARY TABLE LowStockResult;
END //
DELIMITER ;
CALL LowStockMedicines();
```

OUTPUT:

```
+------------+------------------+-------+
| MedicineID | MedicineName     | Stock |
+------------+------------------+-------+
| 104        | Cough Syrup      | 60    |
| 106        | Antacid Gel      | 45    |
| 107        | ORS              | 90    |
| 109        | Metformin        | 80    |
| 110        | Antiseptic Cream | 50    |
+------------+------------------+-------+
5 rows in set
```